

Public	OBJECTIF PÉDAGOGIQUE
Ingénieur 1 (Bac+3)	OBJECTIF PÉDAGOGIQUE : Comprendre la culture DevOps et ses pratiques clés : intégration continue, livraison continue et infrastructure as code. Être capable de containeriser une application avec Docker, de construire un pipeline CI/CD avec GitHub Actions et de déployer sur un environnement cloud ou serveur. Objectifs Opérationnels : À la fin du module, les apprenants seront capables de : - Comprendre la culture DevOps et le mouvement CALMS - Maîtriser Git en équipe : branching strategies, merge requests, code review - Containeriser une application avec Docker - Orchestrer des services avec Docker Compose - Construire un pipeline CI/CD complet avec GitHub Actions - Introduire Kubernetes pour l'orchestration (notions) - Mettre en place le monitoring et les alertes d'une application
Durée	
4 demi-journées	
Prérequis	
OS B2 Git maîtrisé Linux ligne de commande	
Modalités d'évaluation	PROGRAMME CULTURE ET PRINCIPES DEVOPS - Origine du DevOps : silos Dev vs Ops, mouvement CALMS - Les trois voies DevOps : flux, feedback, apprentissage continu - Cycle de vie DevOps : Plan, Code, Build, Test, Release, Deploy, Operate, Monitor - DevSecOps : intégration de la sécurité dans le pipeline - Métriques DORA : deployment frequency, lead time, MTTR, change failure rate GIT AVANCÉ ET COLLABORATION - Branching strategies : Git Flow, GitHub Flow, Trunk Based Development - Pull Requests / Merge Requests : workflow de revue de code - Hooks Git : pre-commit, pre-push (husky, lint-staged) - Tags, releases et versioning sémantique (SemVer) - Gestion des conflits avancée : rebase interactif, cherry-pick DOCKER ET CONTENEURISATION - Architecture Docker : daemon, client, registre, image, conteneur - Dockerfile : instructions FROM, RUN, COPY, ENV, EXPOSE, CMD - Build et optimisation d'images : multi-stage builds, .dockerignore - Gestion des conteneurs : run, ps, logs, exec, stop, rm - Volumes et réseaux Docker : persistance et communication inter-conteneurs DOCKER COMPOSE ET ENVIRONNEMENTS - docker-compose.yml : services, volumes, réseaux, depends_on - Variables d'environnement : .env et override - Compose pour le développement local : hot-reload, debugger
TP pipeline CI/CD + QCM	
Modalités pédagogiques	
Cours TP sur machine	
Matériel et logiciel	
PC + Docker Git / GitHub Linux	• Compose pour les tests d'intégration • TP : stack complète (app + base de données + reverse proxy)
Modalités particulières	
Classe Inversée	
Oui Non	

► Compétences (*)	PROGRAMME (SUITE)
Compétence : Containeriser une app et automatiser son pipeline CI/CD Critère : Pipeline fonctionnel, app déployée en auto	CI/CD AVEC GITHUB ACTIONS • Concepts CI/CD : intégration, livraison et déploiement continus • GitHub Actions : workflows, jobs, steps, runners • Déclencheurs : push, pull_request, schedule, workflow_dispatch • Actions du marketplace : actions/checkout, setup-node, docker/build-push • TP : pipeline complet (lint → test → build → push image → deploy) INFRASTRUCTURE AS CODE (NOTIONS) • Pourquoi l'IaC ? Reproductibilité, versioning, automatisation • Terraform : providers, ressources, variables, état (notions) • Ansible : playbooks, inventaires, modules (notions) • Configuration d'un serveur Linux via Ansible • Comparatif IaC : Terraform vs Ansible vs CloudFormation KUBERNETES — INTRODUCTION • Architecture Kubernetes : cluster, node, pod, service, deployment • kubectl : commandes essentielles • Manifestes YAML : Pod, Deployment, Service, ConfigMap, Secret • Namespaces et gestion des ressources • Minikube / k3s : cluster local pour les tests MONITORING ET OBSERVABILITÉ • Les trois piliers de l'observabilité : logs, métriques, traces • Stack ELK : Elasticsearch, Logstash, Kibana (notions) • Prometheus + Grafana : métriques et dashboards • Alerting : règles d'alerte, canaux (Slack, email) • TP : mise en place d'un tableau de bord applicatif TRAVAUX PRATIQUES ET ÉVALUATION • TP noté : pipeline CI/CD complet pour une application web • Étapes : tests, build Docker, push registry, déploiement auto • QCM DevOps : Docker, Git, CI/CD, Kubernetes • Correction collective et bilan du module
► Niveau visé	
Intermédiaire	
► Outils utilisés	
Docker GitHub Actions Linux	
► Contrôle des acquis	
TP pipeline CI/CD QCM final	